Solving Equations
Numerical Methods

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of HOUSTON | ECE

Solving Equations

Solving Equations Algebraically

Solving Quadratic Equations

Solutions as Roots

Solving Equations Algebraically

- We often require the dependent variable x that satisfies an equation

$$y = f(x)$$

where the function f and y are known

- This can be accomplished by calculating the inverse function:

$$x = f^{-1}(y)$$

Implementing Solutions

- Consider the equation: $y = \sin(x - e)$

$$\sin^{-1}(y) = x - e \qquad f(z) = \sin(z - e)$$

$$\sin^{-1}(y) + e = x \qquad f^{-1}(z) = \sin^{-1}(z) + e$$

- The inverse function $f^{-1}(z) = \sin^{-1}(z) + e$ could be implemented as a series:

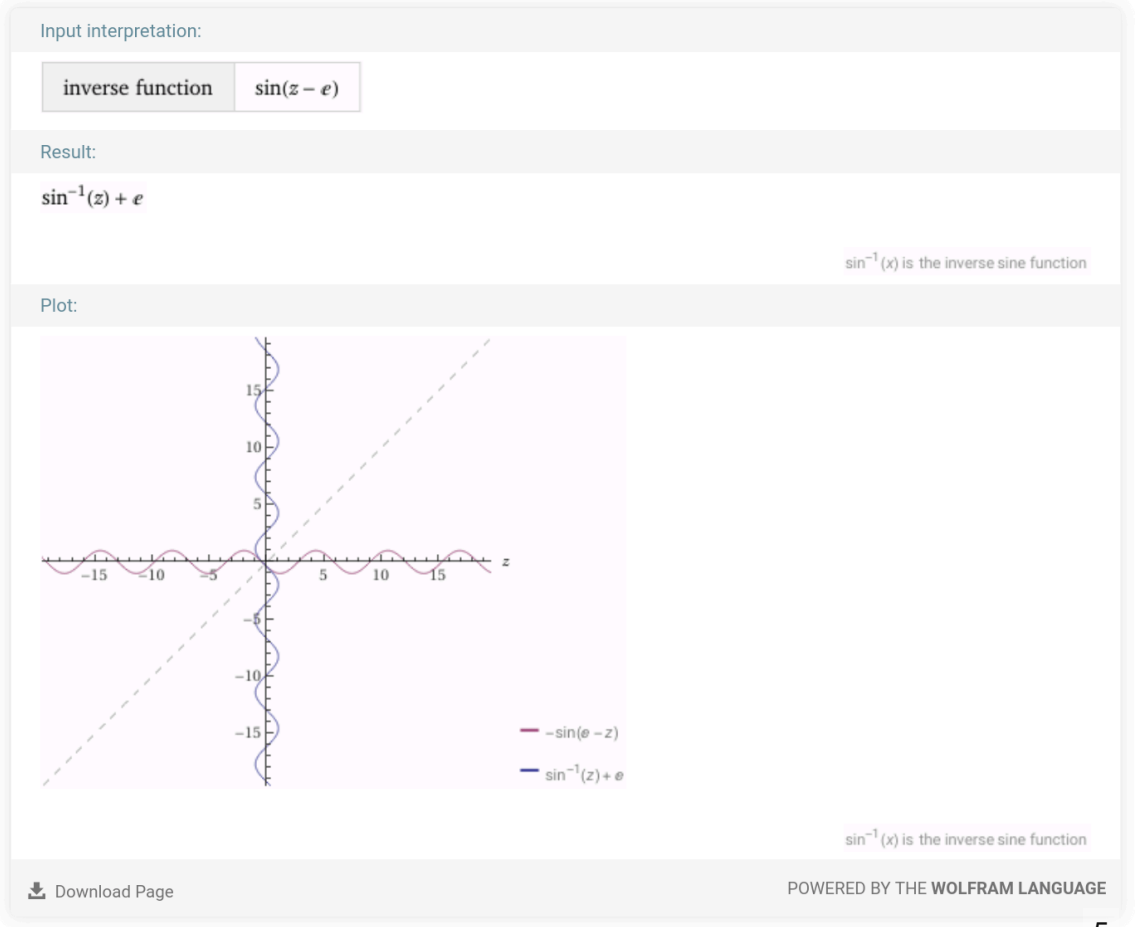
$$\sin^{-1}(x) = x + \frac{x^3}{6} + \frac{3x^5}{40} + O\left(x^6\right) \longrightarrow f^{-1}(z) \approx x + \frac{x^3}{6} + \frac{3x^5}{40} + e$$

Computer Algebra Systems

- How can we **get** the inverse formulation: $f(z) = \sin(z - e) \rightarrow f^{-1}(z) = \sin^{-1}(z) + e$?
- Computer Algebra Systems (CAS) can get us there:
  - Mathematica/Alpha (wolframalpha.com)
  - Symbolic Math Toolbox (MATLAB)
  - SymPy (sympy.org)

```
1 from sympy import *
2
3 y, z = symbols("y z")
4 solve(sin(z - exp(1)) - y, z)
```

```
[{z: asin(y) + E}, {z: -asin(y) + E + pi}]
```



Numerical Solutions

- Consider functions without inverses

  (the vast majority of them)

$$y = \sin\left(z^2 - e^z\right)$$

- Wolfram Alpha cannot calculate the inverse but can generate a graph
- This is done by expressing the equation as a *root*:

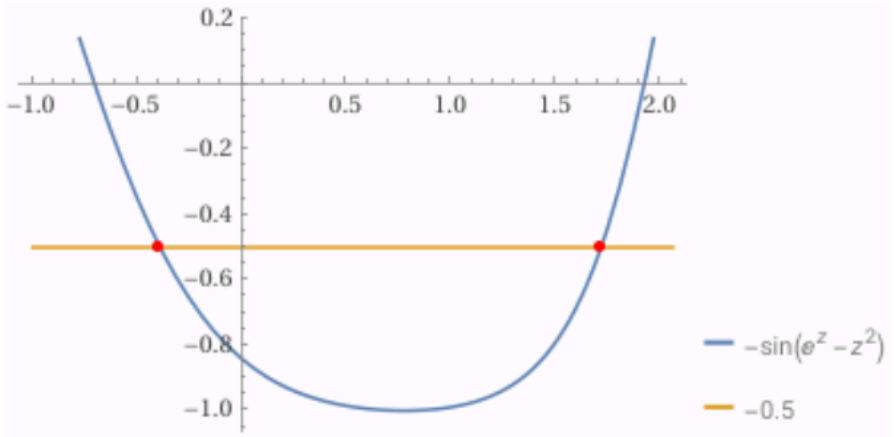
$$\sin\left(z^2 - e^z\right) - y = 0$$

- Finding the roots of this equation for a given y gives the solution x

Root-Finding Formulas

- Find the solution to $y = \sin\left(z^2 - e^z\right)$ for $y = -0.5$

$$\boldsymbol{z = -0.3909}$$



- Express the special functions as series:

$$\sin(z) \approx z \qquad e^z \approx 1 + z + \frac{z^2}{2}$$

$$\sin\left(z^2 - e^z\right) \approx \frac{z^2}{2} - z - 1$$

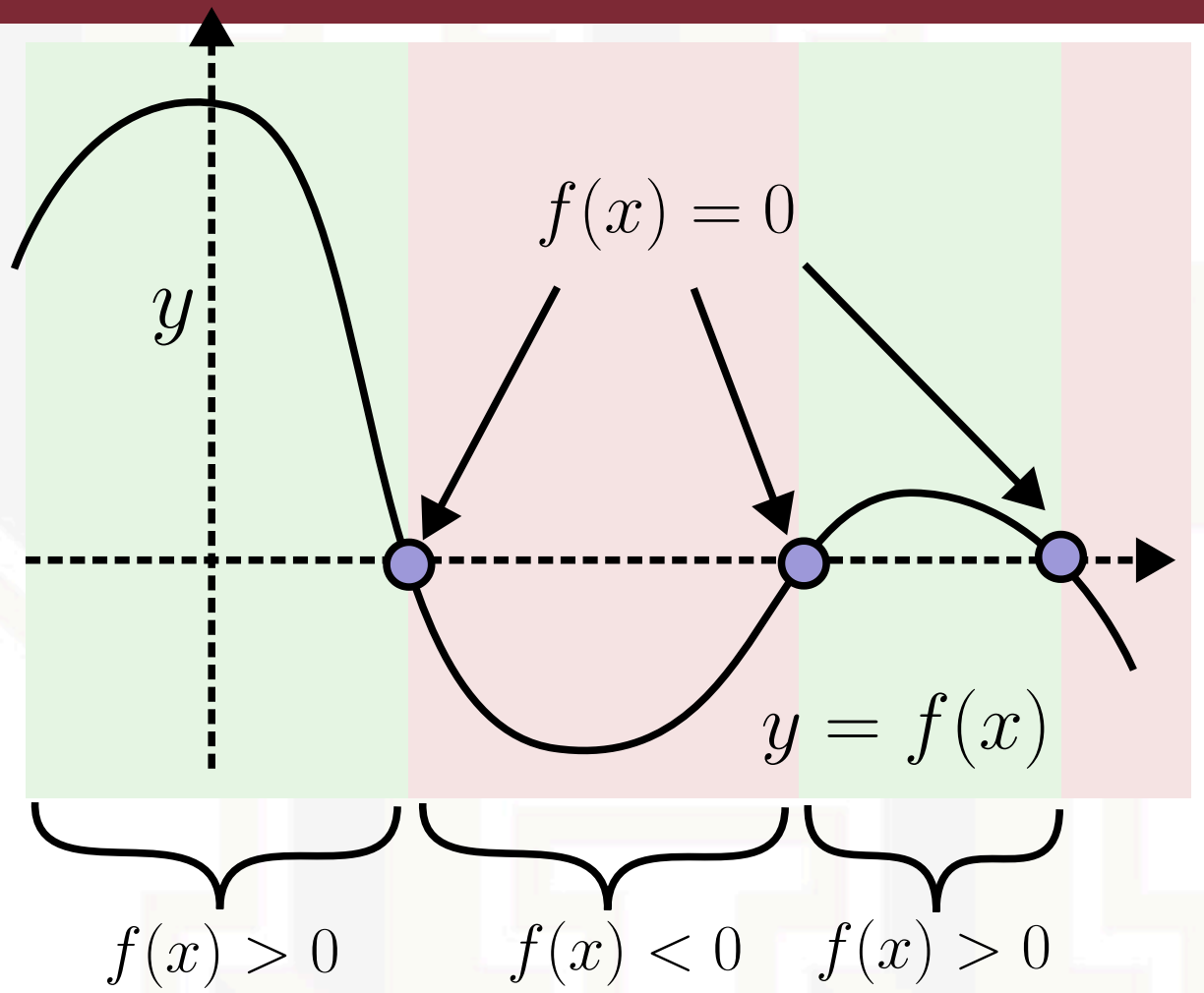
$$\frac{z^2}{2} - z - 1 - y = 0 \qquad \frac{1}{2}z^2 - z - 0.5 = 0$$

$$z = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \qquad z = \frac{1 \pm \sqrt{(-1)^2 - 4(0.5)(-0.5)}}{2(0.5)} = [\,-0.4142,\ 2.414\,]$$

$$\epsilon_r \approx 5.9\%$$

Simple Roots

- Consider an interval $[a, b]$ of a function $f(x) \in \mathbb{R}$ where $f(a)$ and $f(b)$ have opposite signs:
$$f(a) \cdot f(b) < 0$$
- If $f(x)$ is continuous on $[a, b]$, then there exists a number $c \in [a, b]$ such that $f(c) = 0$
- The value $x = c$ is a *simple root* of f



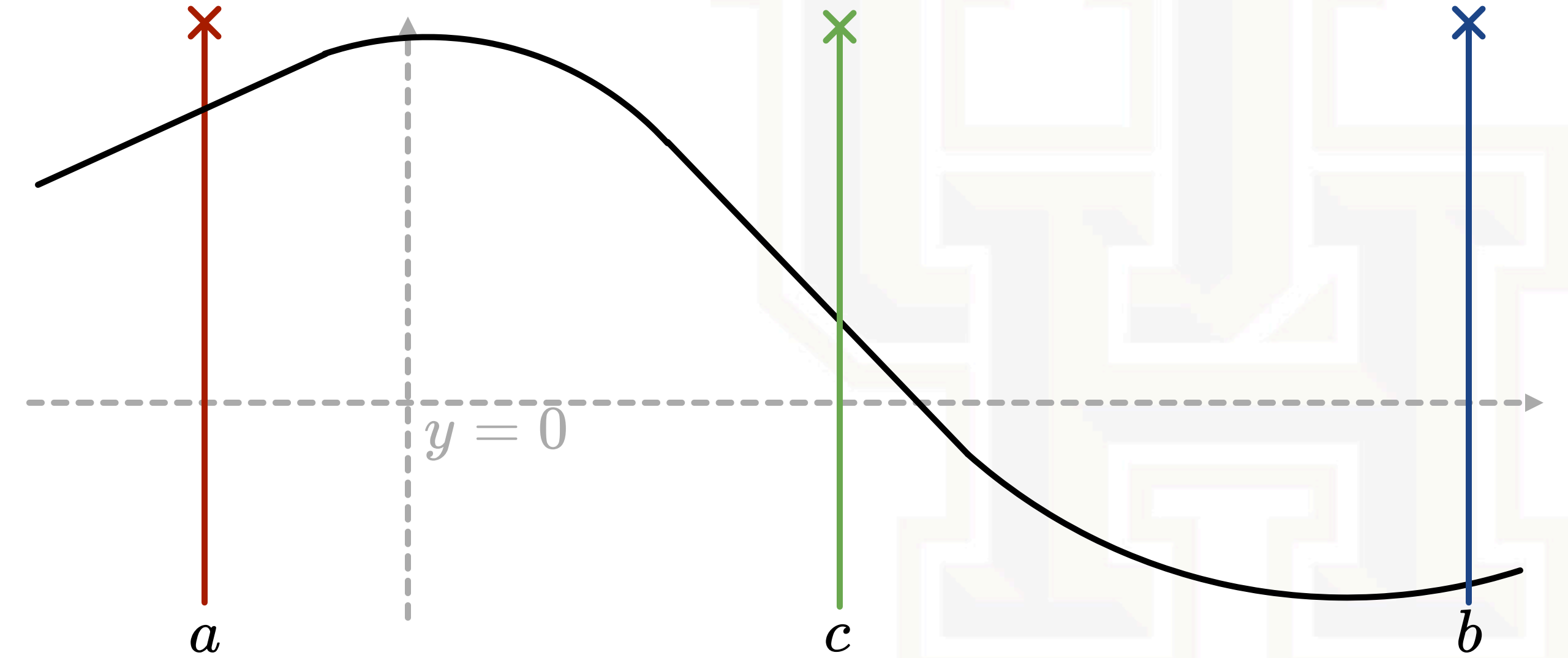
Bisection Method

Binary Searches for Roots

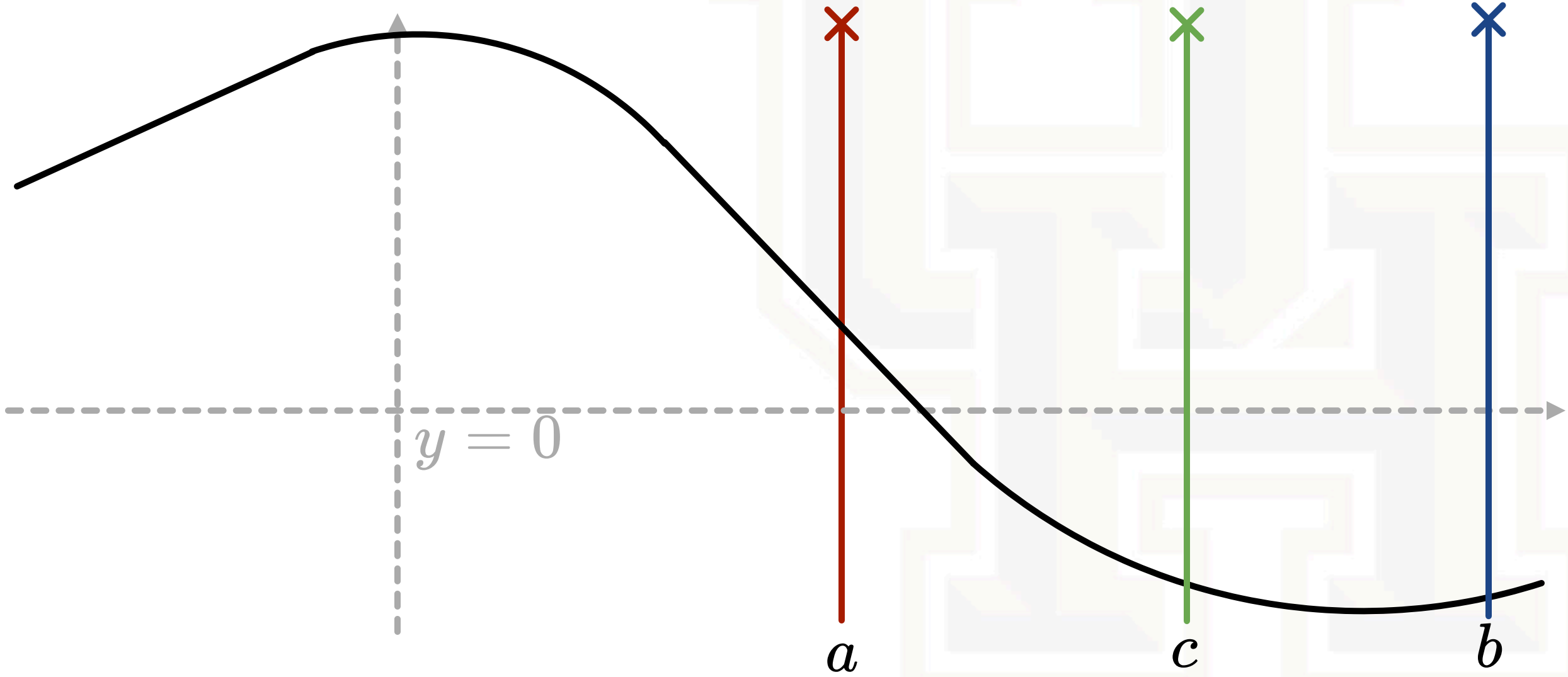
Bisection Algorithm

Error

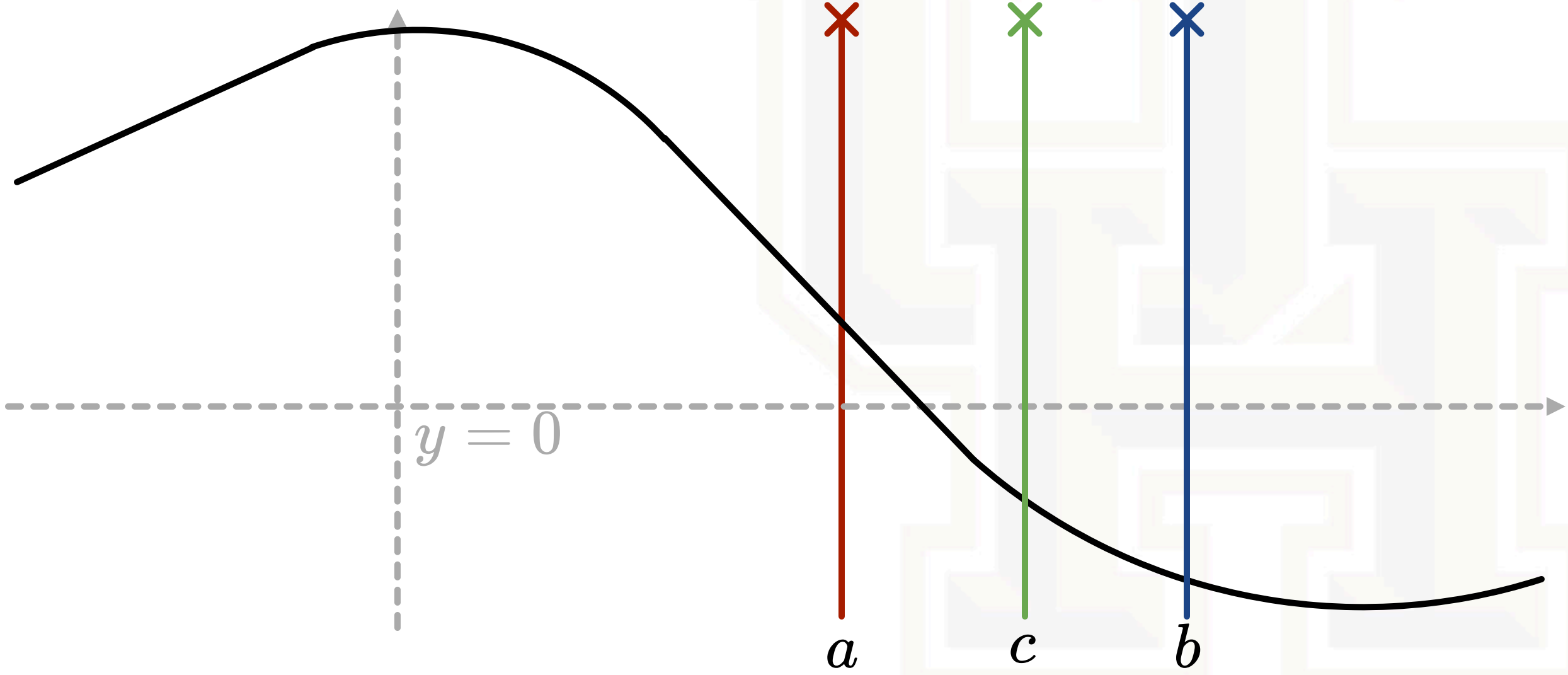
Bisection Method



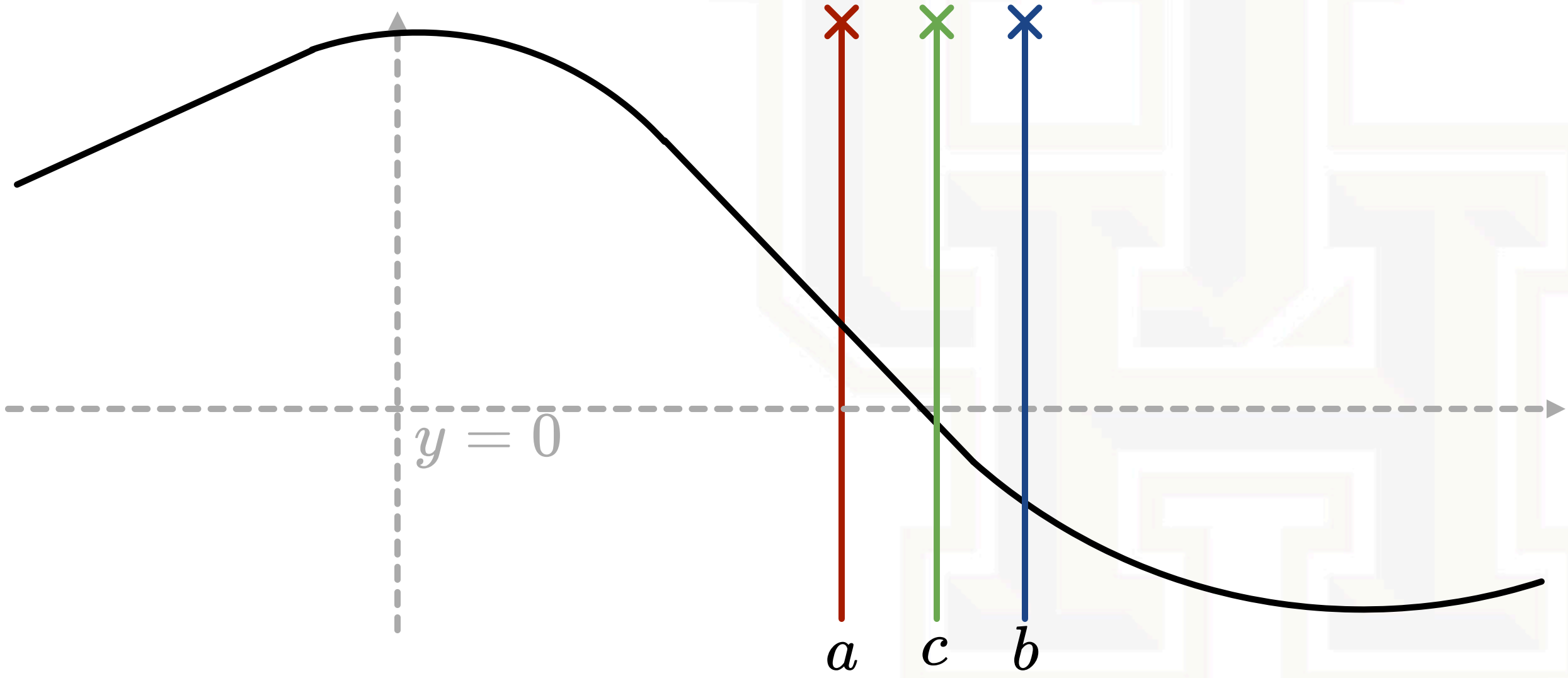
Bisection Method



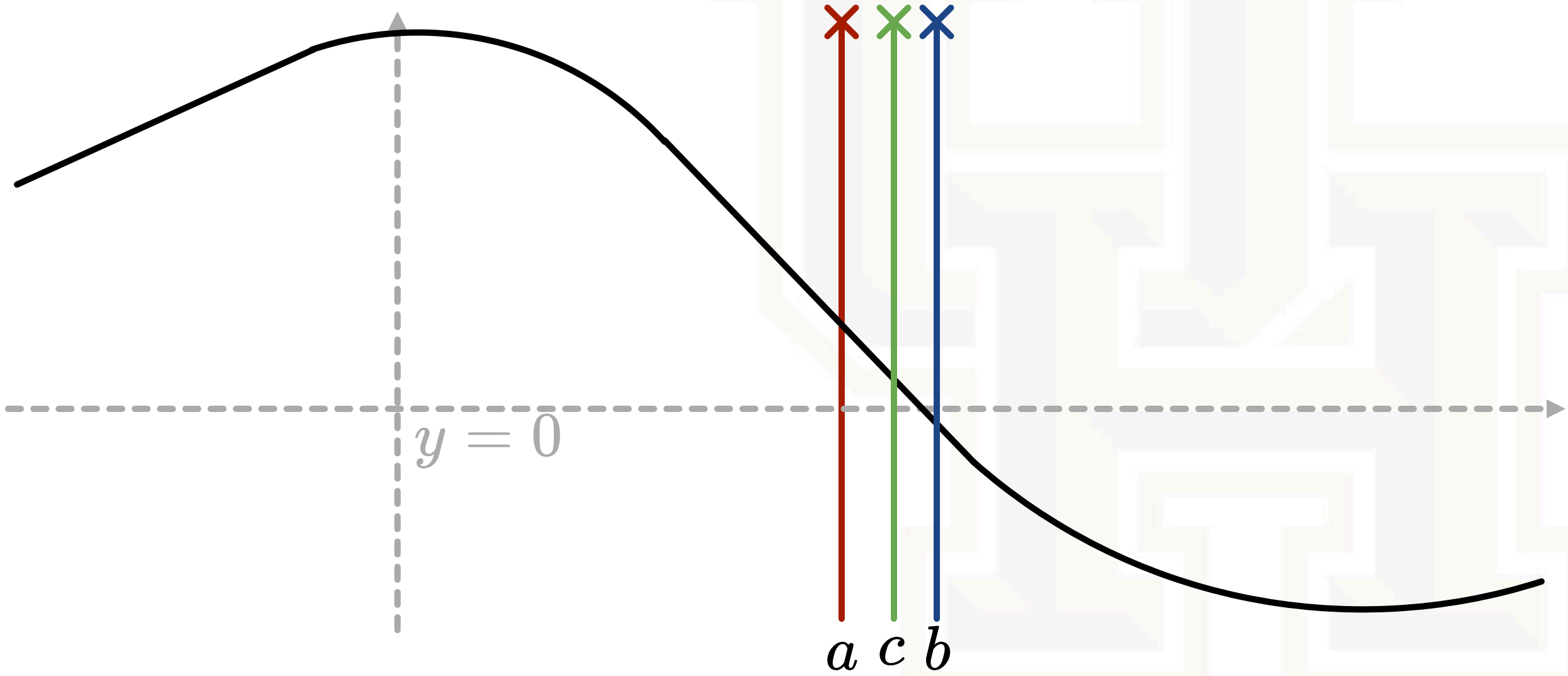
Bisection Method



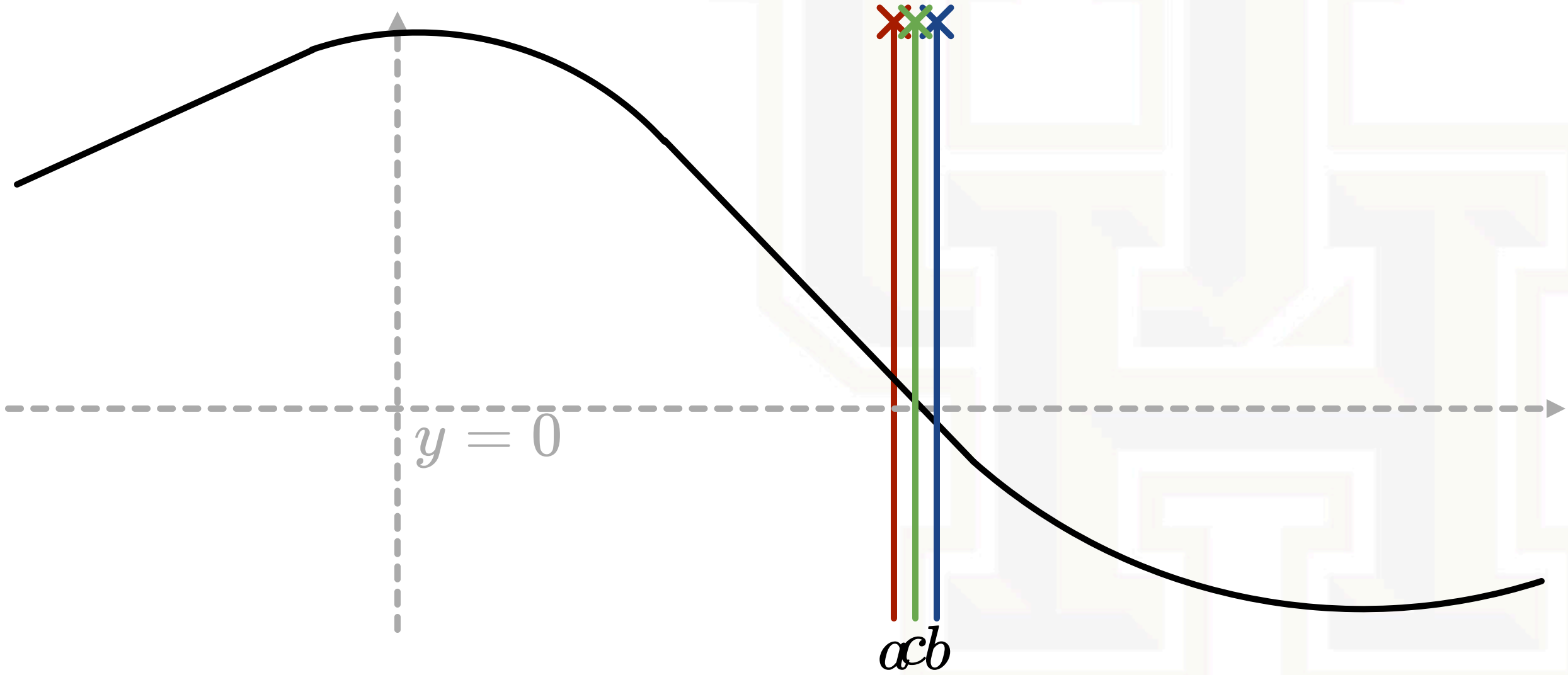
Bisection Method



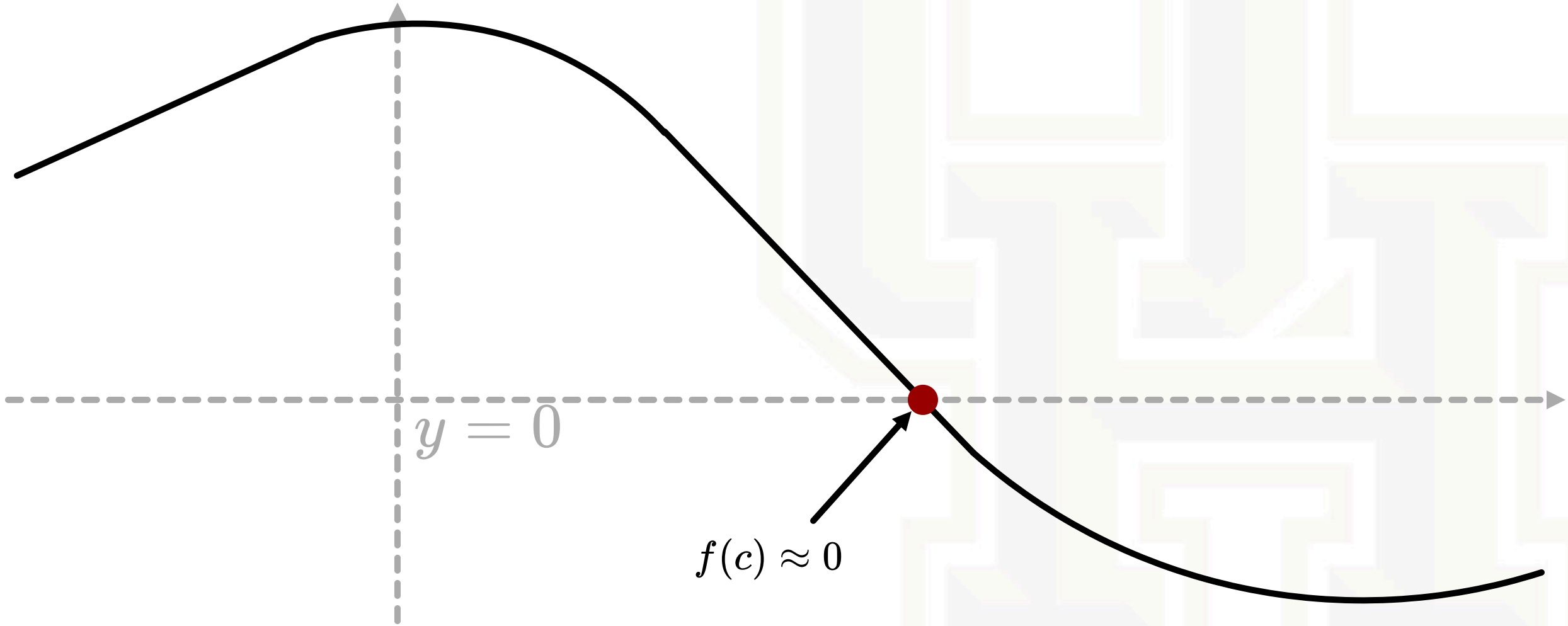
Bisection Method



Bisection Method



Bisection Method



Bisection Method Evaluation

- Calculate the solution to $y = \sin\left(x^2 - e^x\right)$ for $y = -0.5$ given the interval $[-1.0, -0.1]$ out to 4 digits of precision

$$\sin\left(x^2 - e^x\right) = -0.5 \longrightarrow \sin\left(x^2 - e^x\right) + 0.5 = 0$$

a	f(a)	b	f(b)	c	f(c)
−1.000	1.091	−0.1000	−0.2801	−0.5500	0.2290
−0.5500	0.2290	−0.1000	−0.2801	−0.3250	−0.07851
−0.5500	0.2290	−0.3250	−0.07851	−0.4375	0.06122
−0.4375	0.06122	−0.3250	−0.07851	−0.3813	−0.01206
−0.4375	0.06122	−0.3813	−0.01206	−0.4094	0.0237015
−0.4094	0.0237015	−0.3813	−0.01206	−0.3954	0.005665
−0.3954	0.005665	−0.3813	−0.01206	−0.3884	−0.003190
−0.3954	0.005665	−0.3884	−0.003190	−0.3919	0.001224
−0.3919	0.001224	−0.3884	−0.003190	−0.3902	−0.0009235
−0.3919	0.001224	−0.3902	−0.0009235	−0.3911	0.0002126
−0.3911	0.0002126	−0.3902	−0.0009235	−0.3907	−0.0002925
−0.3911	0.0002126	−0.3907	−0.0002925	−0.3909	−0.00004002

Bisection Method Algorithm

```
1  // implement the bisection method where:
2  // f is a function pointer that takes a parameter x and returns a value
3  // a and b are brackets bounding a root
4  // N is the maximum number of iterations
5  // return an approximation to the root bounded by a and b
6  double bisection(double (*f)(double), double a, double b, int N){
7    float c;
8
9    return c; // return the last approximation to the root
10 }
```

Bisection Method Algorithm

```
1  // implement the bisection method where:
2  // f is a function pointer that takes a parameter x and returns a value
3  // a and b are brackets bounding a root
4  // N is the maximum number of iterations
5  // return an approximation to the root bounded by a and b
6  double bisection(double (*f)(double), double a, double b, int N){
7    float c;
8    for(int i = 0; i < N; i++){ // iterate N times
9
10   }
11   return c; // return the last approximation to the root
12 }
```

Bisection Method Algorithm

```
1  // implement the bisection method where:
2  // f is a function pointer that takes a parameter x and returns a value
3  // a and b are brackets bounding a root
4  // N is the maximum number of iterations
5  // return an approximation to the root bounded by a and b
6  double bisection(double (*f)(double), double a, double b, int N){
7    float c;
8    for(int i = 0; i < N; i++){ // iterate N times
9      c = (a + b) / 2.0; // find midpoint to approximate f(c) = 0
10     if(f(c) == 0) return c; // if f(c) actually IS zero, return it (rare)
11     if(f(a) * f(c) > 0) a = c; // if f(c) has the same sign as f(a), move a
12     else b = c; // otherwise move b
13   }
14   return c; // return the last approximation to the root
15 }
```

Bisection Method - Example

- Calculate the solution to $y = \sin\left(x^2 - e^x\right)$ for $y = -0.5$ given the interval $[-1.9, 1.7]$

```
1  double func(double z) {
2      return = sin(z*z - exp(z));
3  }
4
5  int main(int argc, char** argv) {
6      int N = 100;
7
8      double result = bisection(func, -1.9, 1.7, N);
9
10 }
```

Bisection Method Theorem

If the bisection method is applied to a continuous function f on an interval $[a, b]$, where $f(a)f(b) < 0$, then after n steps an approximate root will be computed with an error at most $\epsilon = \frac{b-a}{2^n}$.

- Each iteration computes the root with one additional binary bit of precision

- How many iterations are required to guarantee an error less than ϵ'?

$$\frac{b-a}{2^n} < \epsilon'$$

$$b - a < 2^n \epsilon'$$

$$\log(b-a) < n \log 2 + \log \epsilon'$$

$$\log(b-a) - \log \epsilon' < n \log 2$$

$$n > \frac{\log(b-a) - \log \epsilon'}{\log 2}$$

$$n = \left\lceil \frac{\log(b-a) - \log \epsilon'}{\log 2} \right\rceil$$

Bisection Method Characteristics

- Very robust - guaranteed to converge to a root as long as $f(a)f(b) < 0$
- Does not work for multiple roots when $f(a)f(b) \geq 0$
- Brute-force binary search reduces error by $O(2^n)$ every iteration
  - This is equivalent to 1 bit per iteration
  - ≈ 23 iterations for `float`, ≈ 53 iterations for `double`